AutoBI Project Explanation

A beginner-friendly guide to the code, workflow, and interview story

What it is: a local browser-based analytics application. FastAPI processes CSV/Excel uploads; Pandas and NumPy analyze them; Plotly.js presents charts. "AI insights" are deterministic data rules, not a call to a generative AI service.

1. Project overview and workflow

AutoBI turns a tabular file into a dataset profile, optional cleaned version, exploratory charts, KPI cards, data-driven insights, and an evaluated numeric prediction. The usual flow is: upload -> profile -> optionally clean -> explore (EDA) -> filter KPIs -> review insights/prediction -> optionally download cleaned data.

The browser keeps the selected upload in per-tab IndexedDB so it can restore the workspace after a reload. For analysis requests, the browser sends the file and selected options to FastAPI again. The backend does not keep uploaded data in global server memory.

2. Folder structure and important files

```
Project root: README.md, run.bat, test.bat, configure_database.py
Backend/app/: main.py, analysis.py, cleaning.py, eda.py, kpi.py, ai_analysis.py,
database.py
Backend/: requirements.txt, requirements-dev.txt, requirements-lock.txt, .env.example
Backend/tests/: API, cleaning, EDA, KPI, and AI-analysis tests
Frontend/: index.html, script.js, style.css
```

README.md explains setup and design. Backend/app/main.py connects the HTTP routes to analysis functions. Frontend/index.html defines the interface; script.js handles uploads, requests, rendering, and navigation; style.css controls its responsive look. run.bat starts the application. test.bat runs project checks.

3. Frontend, FastAPI APIs, and communication

The frontend is HTML, CSS, and JavaScript. FastAPI serves the same interface through Jinja2 and serves its static files; the browser can also be hosted separately for development. JavaScript sends multipart/form-data requests with the file and options, then receives JSON data or a streamed download. Plotly.js turns chart specifications into interactive charts.

```
GET  /health                    service and optional MySQL status
POST /api/analyze               profile, EDA, insights, dashboard charts, default
prediction
POST /api/clean                 before/after counts and cleaned preview
POST /api/clean/download/csv    cleaned CSV download
POST /api/clean/download/xlsx   cleaned Excel download
POST /api/eda                   automatic EDA report
POST /api/eda/cleaned           EDA after selected cleaning options
POST /api/eda/chart             validate/build a selected chart
POST /api/kpi/dashboard         filtered KPI cards and charts
POST /api/predictions           evaluate a selected numeric target
```

Typical cycle: the user selects a file; script.js posts it to /api/analyze; main.py validates and parses it; analysis modules return JSON; JavaScript renders the metrics, tables, insight cards, and Plotly charts. Cleaning and filters use additional requests that include the same file and options.

4. Python libraries and why they are used

- FastAPI: HTTP routes and request handling. Uvicorn: runs the ASGI web application.
- Pandas: reads CSV/Excel and performs tabular profiling, cleaning, grouping, and statistics. NumPy: numerical calculations and safe sampling.
- scikit-learn: median imputation, train/test splitting, linear regression, and evaluation metrics.
- python-multipart: reads uploaded files and form fields. Jinja2: renders the shared HTML template.
- openpyxl and xlrd: read Excel formats; openpyxl also writes .xlsx exports.
- SQLAlchemy: defines the metadata table and database sessions. PyMySQL: MySQL driver. python-dotenv: loads Backend/.env.
- Pydantic (included with FastAPI): validates request fields and chart selections.

Browser libraries: Bootstrap (layout/components) and Plotly.js (interactive charts) are loaded from CDNs. They need a network connection unless supplied another way.

5. Upload, analysis, cleaning, EDA, charts, and KPIs

Uploads accept CSV, XLSX, and XLS; the backend rejects unsupported, empty, malformed, oversized, or over-limit data. Limits are 25 MiB and 100,000 parsed rows. The overview reports rows, columns, missing cells, duplicates, column types, numeric summaries, and a ten-row preview.

Cleaning is optional and does not change the source upload. Options are keep missing values (default), fill numeric values by mean or median, fill text/boolean values by mode, drop rows with missing cells, remove duplicates, and standardize column names. The UI shows before/after counts and preview. Cleaned CSV and XLSX files can be downloaded.

EDA reports numeric descriptive statistics and suitable automatic charts such as distributions, category counts, box summaries, correlations, scatter relationships, and date trends. Users may request a supported chart type (histogram, bar, pie, scatter, line, or box) for selected columns. Data-dependent limits prevent excessively large charts; unsuitable selections return an explanation.

The KPI dashboard applies date and category filters consistently to its cards and charts. It reports row counts and supported measures; sales/profit or order/customer/product metrics depend on matching column names. If those are absent, it uses suitable generic numeric summaries. It does not guess currency or units.

6. Day 5 predictions and AI insights

Insights are generated locally from data patterns: missing-value concentrations, strong Pearson correlations, numeric outliers (1.5 x IQR), dominant categories, and date/numeric trends. Each insight includes a suggested action. These are rule-based, repeatable explanations; no external AI or LLM service is called, and correlation is not proof of cause.

Prediction uses scikit-learn linear regression. The user selects a numeric target; other varying numeric columns are features, with missing feature values filled using medians learned from training rows. At least eight finite target values and usable numeric features are needed. A deterministic 75/25 train/holdout split reports MAE, RMSE, and R-squared when defined, with actual-versus-predicted values. This evaluates unseen rows; it is not a forecast of future dates. Text features are not used.

7. MySQL, SQLAlchemy, and persistence

MySQL is optional. When DATABASE_URL is configured in Backend/.env, SQLAlchemy creates/uses the dataset_analyses table and stores analysis metadata: filename, row/column counts, missing and duplicate counts, column summary, and timestamp. The uploaded dataset rows are not stored in MySQL. Without a configured database, analysis still works; a failed metadata save is logged and shown as a warning.

configure_database.py prompts for connection details and writes Backend/.env; keep credentials private. In contrast, the browser IndexedDB workspace stores the current upload locally in that browser profile for reload recovery.

8. Reports, exports, and application startup

Implemented exports are cleaned-data CSV and Excel downloads. The Reports navigation/page is a placeholder: shareable or generated reports are planned, but report generation/export is not implemented. This PDF is documentation, not an AutoBI report feature.

For the recommended Windows setup, create the virtual environment and install the locked packages once, then start from the project root. run.bat checks the environment and port, then starts Uvicorn with reload. Default address: http://127.0.0.1:8000/. FastAPI docs: /docs. The app can also be tested with test.bat.

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r Backend\requirements-lock.txt
run.bat
```

9. Important functions and their roles

- Backend/app/main.py: read_dataset validates/parses uploads; analyze assembles the overview response; clean and download_cleaned_dataset return cleaned previews/downloads; eda, cleaned_eda, selected_eda_chart, kpi_dashboard, and predictions expose analysis APIs.
- Backend/app/analysis.py: analyze_dataframe calculates profile counts, column details, summary, preview, and basic chart data.
- Backend/app/cleaning.py: clean_dataframe applies selected operations to a copy; summarize_dataframe, dataframe_preview, and prepare_excel_export support the UI and exports.
- Backend/app/eda.py: build_eda_report creates bounded statistics/chart specs; build_dashboard_visualizations selects overview charts; build_selected_chart validates a user-selected chart.
- Backend/app/kpi.py: build_kpi_dashboard filters the current dataframe and builds explainable KPI cards and chart data.
- Backend/app/ai_analysis.py: build_ai_analysis/build_automatic_insights create rule-based insights; prediction_targets and build_prediction_report select and evaluate regression targets.
- Backend/app/database.py: DatasetAnalysis maps metadata to SQLAlchemy; persist_analysis writes it when MySQL is configured.
- Frontend/script.js: analyzeUploadedFile uploads and renders the overview; cleanDataset and downloadCleanedDataset handle cleaning; renderEdaReport/buildSelectedChart show EDA; refreshKpiDashboard renders filtered KPI results; runPrediction sends a selected target; saveWorkspaceState/restoreWorkspaceOnStartup use IndexedDB.

10. How to explain it in an interview

?AutoBI is a browser-based BI tool built with a JavaScript frontend and a FastAPI backend. A user uploads CSV or Excel data; Pandas and NumPy profile it, optional cleaning preserves the original, and EDA/KPI modules return chart-ready data for Plotly. It also provides deterministic data insights and a held-out linear-regression evaluation. MySQL is optional and stores analysis metadata only. The Reports page is currently a placeholder.?

Good points to mention: clear separation between routes and analysis modules; upload and chart limits; cleaning is opt-in; cleaned data is recalculated from the original upload and selected options; database persistence degrades gracefully; and the ML result is an evaluation, not a time-series forecast.